

Project: ISO JTC1/SC22/WG21: Programming Language C++  
Doc No: WG21 **P0600R1**  
Date: 2017-11-09  
Reply to: Nicolai Josuttis (nico@josuttis.de)  
Audience: LEWG, LWG  
Prev. Version: P0600R0

# `[[nodiscard]]` in the Library, Rev1

## Updates for Version R1:

- added `empty()`
- added `launder()` in wording
- no `[[nodiscard]]` for C functions (removed `malloc()`)
- require also to add `[[nodiscard]]` in the definition
- fixed reference paper and section numbering
- reason for not having a feature test macro

C++17 introduced the `[[nodiscard]]` attribute.

The question is, where to apply it now in the standard library.

We suggest a conservative approach:

It should be added where:

- For existing API's
  - not using the return value always is a “huge mistake” (e.g. always resulting in resource leak)
  - not using the return value is a source of trouble and easily can happen (not obvious that something is wrong)
- For new API's (not been in the C++ standard yet)
  - not using the return value is usually an error.

It should not be added when:

- For existing API's
  - not using the return value is a possible/common way of programming at least for some input
    - for example for `realloc()`, which acts like `free` when the new size is 0
  - not using the return value makes no sense but doesn't hurt and is usually not an error (e.g., because programmers meant to ask for a state change).
  - it is a C function, because their declaration might not be under control of the C++ implementation

For example:

| Function                                      | <code>[[nodiscard]]</code> ? | Remark                                                                                                                                                                                      |
|-----------------------------------------------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>malloc()</code>                         | no                           | expensive call, usually not using the return value is a resource leak. However, a C function.                                                                                               |
| <code>realloc()</code>                        | no                           | <code>realloc()</code> with new size 0 acts like <code>free()</code>                                                                                                                        |
| <code>async()</code>                          | yes                          | not using the return value makes the call synchronous, which might be hard to detect.                                                                                                       |
| <code>launder()</code>                        | yes                          | new API, where not using the return value makes no sense, because <code>launder()</code> does not white-wash. It just the return value allows to use the corresponding data “white washed”. |
| <code>allocate()</code>                       | yes                          | same as <code>malloc()</code>                                                                                                                                                               |
| <code>unique_ptr::release()</code>            | no                           | Titus: at Google 3.5% of calls would fail, but analysis showed that it was correct (but weird ownership semantics). See reflector email.                                                    |
| <code>printf()</code> , <code>sprint()</code> | no                           | too many code not using the return value (which also is not always necessary according to programming logic)                                                                                |
| <code>top()</code>                            | no                           | not very useful, but no danger and such code might exist                                                                                                                                    |
| <code>empty()</code>                          | yes                          | doesn't hurt, but (as reported by multiple parties) not using the return value often is an error, because programmers meant <code>clear()</code>                                            |

So, `[[nodiscard]]` should not signal bad code if this

- a) *can* be useful not to use the return value
- b) is common not to use the return value
- c) doesn't hurt and probably no state change was meant that doesn't happen

Or as Andrew Tomazos wrote in an email:

1. You almost never want to discard the return value. (Using the return value is almost always an essential part of the interface of the given function.)

and

2. People do sometimes discard the return value of that function by accident. (They do so because they misunderstand the interface of the function, incorrectly thinking it doesn't return a value, or the return value is non-essential extra information.)

As a result, initially I see the following modifications for C++17:

add `[[nodiscard]]` to:

- `async()`
- `allocate()`, `operator new`
- `launder()`, `empty()`

## Proposed Wording

(All against N4700)

### `async()`:

#### 33.6.1 Overview [futures.overview]

```
template <class F, class... Args>
    [[nodiscard]] future<result_of_t<decay_t<F>(decay_t<Args>...)>>
    async(F&& f, Args&&... args);
template <class F, class... Args>
    [[nodiscard]] future<result_of_t<decay_t<F>(decay_t<Args>...)>>
    async(launch policy, F&& f, Args&&... args);
```

Also in the corresponding definitions in **33.6.8 Function template `async`** [futures.async]

```
template <class F, class... Args>
    [[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(F&& f, Args&&... args);
template <class F, class... Args>
    [[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(launch policy, F&& f, Args&&... args);
```

### `launder()`:

#### 21.6.1 Header `<new>` synopsis [new.syn]:

```
template <class T> [[nodiscard]] constexpr T* launder(T* p) noexcept;
```

Also in the corresponding definitions in **21.6.4 Pointer optimization barrier** [ptr.launder]:

```
template <class T> [[nodiscard]] constexpr T* launder(T* p) noexcept;
```

## `allocate()`:

### 17.5.3.5 Allocator requirements `[allocator.requirements]`

§9, in the example:

```
[[nodiscard]] T* allocate(std::size_t n);
```

### 23.10.8 Allocator traits `[allocator.traits]`

```
static [[nodiscard]] pointer allocate(Alloc& a, size_type n);  
static [[nodiscard]] pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
```

Also in the corresponding definitions in **23.10.8.2 Allocator traits static member functions** `[allocator.traits.members]`.

### 23.10.9 The default allocator `[default.allocator]`

```
[[nodiscard]] T* allocate(size_t n);
```

Also in the corresponding definition in **23.10.9.1 allocator members** `[allocator.members]`.

### 23.12.2 Class memory\_resource `[mem.res.class]`

```
[[nodiscard]] void* allocate(size_t bytes, size_t alignment = max_align);
```

Also in the corresponding definition in **23.12.2.1 memory\_resource public member functions** `[mem.res.public]`.

### 23.12.3 Class template polymorphic\_allocator `[mem.poly.allocator.class]`

```
[[nodiscard]] T* allocate(size_t n);
```

Also in the corresponding definition in **23.12.3.2 polymorphic\_allocator member functions** `[mem.poly.allocator.mem]`.

### 23.13.1 Header `<scoped_allocator>` synopsis `[allocator.adaptor.syn]`

```
[[nodiscard]] pointer allocate(size_type n);  
[[nodiscard]] pointer allocate(size_type n, const_void_pointer hint);
```

Also in the corresponding definition in **23.13.4 Scoped allocator adaptor members** `[allocator.adaptor.members]`.

## `operator new()`:

### 6.7.4 Dynamic storage duration `[basic.stc.dynamic]`

```
[[nodiscard]] void* operator new(std::size_t);  
[[nodiscard]] void* operator new(std::size_t, std::align_val_t);  
...  
[[nodiscard]] void* operator new[](std::size_t);  
[[nodiscard]] void* operator new[](std::size_t, std::align_val_t);
```

### 21.6.1 Header `<new>` synopsis `[new.syn]`

```
[[nodiscard]] void* operator new(std::size_t size);  
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment);  
[[nodiscard]] void* operator new(std::size_t size, const std::nothrow_t&) noexcept;  
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment,  
                                const std::nothrow_t&) noexcept;
```

```
...
[[nodiscard]] void* operator new[](std::size_t size);
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment);
[[nodiscard]] void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment,
                                   const std::nothrow_t&) noexcept;
...
[[nodiscard]] void* operator new (std::size_t size, void* ptr) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, void* ptr) noexcept;
```

Also in the corresponding definitions in the subsections of **21.6.2 Storage allocation and deallocation** `[new.delete]`.

## `empty()`:

In **24.3.2 Class template `basic_string`** `[basic.string]`:

```
[[nodiscard]] bool empty() const noexcept;
```

Also in the corresponding definitions in **24.3.2.4 `basic_string` capacity** `[string.capacity]`.

In **24.4.2 Class template `basic_string_view`** `[string.view.template]`:

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

Also in the corresponding definitions in **24.4.2.3 Capacity** `[string.view.capacity]`.

In **26.2.4.1 `node_handle` overview** `[container.node.overview]`:

```
[[nodiscard]] bool empty() const noexcept;
```

Also in the corresponding definitions in **26.2.4.4 `node_handle` observers** `[container.node.observers]`.

In **26.3.7.1 Class template `array` overview** `[array.overview]` and

**26.3.9.1 Class template `forward_list` overview** `[forwardlist.overview]` and

**26.3.10.1 Class template `list` overview** `[list.overview]` and

**26.3.11.1 Class template `vector` overview** `[vector.overview]` and

**26.3.12 Class `vector<bool>`** `[vector.bool]` and

**26.4.4.1 Class template `map` overview** `[map.overview]` and

**26.4.5.1 Class template `multimap` overview** `[multimap.overview]` and

**26.4.6.1 Class template `set` overview** `[set.overview]` and

**26.4.7.1 Class template `multiset` overview** `[multiset.overview]` and

**26.5.4.1 Class template `unordered_map` overview** `[unord.map.overview]` and

**26.5.5.1 Class template `unordered_multimap` overview** `[unord.multimap.overview]` and

**26.5.6.1 Class template `unordered_set` overview** `[unord.set.overview]` and

**26.5.7.1 Class template `unordered_multiset` overview** `[unord.multiset.overview]`:

```
[[nodiscard]] bool empty() const noexcept;
```

In **26.6.4.1 `queue` definition** `[queue.defn]` and

**26.6.5 Class template `priority_queue`** `[priority.queue]` and

**26.6.6.1 `stack` definition** `[stack.defn]`:

```
[[nodiscard]] bool empty() const { return c.empty(); }
```

In **27.3 Header `<iterator>` synopsis** [`iterator.synopsis`]:

```
template <class C> [[nodiscard]] constexpr auto empty(const C& c) -> decltype(c.empty());  
template <class T, size_t N> [[nodiscard]] constexpr bool empty(const T (&array)[N]) noexcept;  
template <class E> [[nodiscard]] constexpr bool empty(initializer_list<E> il) noexcept;
```

Also in the corresponding definitions in **27.8 Container access** [`iterator.container`].

In **30.10.7 Class `path`** [`fs.class.path`]

```
[[nodiscard]] bool empty() const noexcept;
```

Also in the corresponding definitions in **30.10.7.4.10 path query** [`fs.path.query`].

In **31.10 Class `template match_results`** [`re.results`]

```
[[nodiscard]] bool empty() const;
```

Also in the corresponding definitions in **31.10.3 `match_results` size** [`re.results.size`]

## Feature Test Macro

No feature test applied because for the caller nothing changed regarding what can be called or not. Only a warning might be emitted.